Université Mohammed V de Rabat
École Nationale Supérieure d'Informatique et d'Analyse des Systèmes

TD : IDM – MDA
Transformation par template avec Acceleo

Modèle → texte (M2T) : Java, SQL, Python, HTML

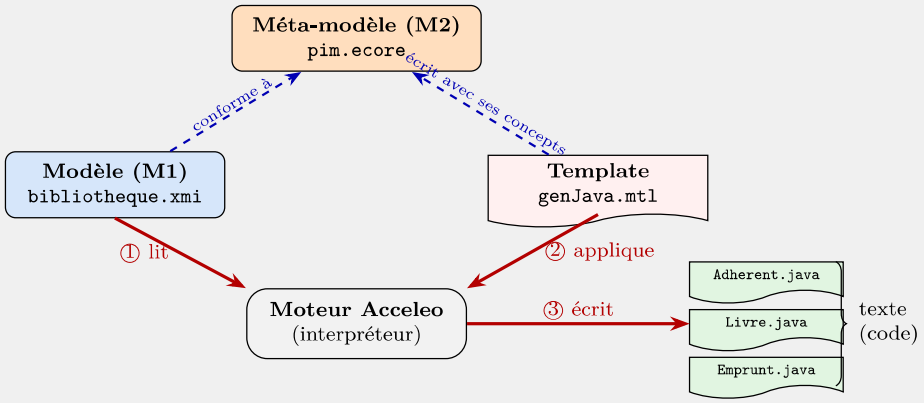
(Énoncé)

Pr.Aissam BERRAHOU
Année universitaire 2026–2027

Objectifs et consignes

1. Expliquer le principe d'une transformation **modèle → texte** (M2T) **par template**.
2. **Lire** un module Acceleo et **prédire** le texte qu'il produit.
3. **Naviguer** dans un modèle avec des expressions OCL (`.`, `->select`, `->exists`, `->size`…).
4. **Écrire** des templates et des queries (Python, SQL, Java, XML) et **corriger** un template erroné.

Le TD suit le **fil rouge du cours** (Lecture 3) : le méta-modèle `pim.ecore` et le modèle de la **bibliothèque** (`bibliotheque.xmi`). Les exercices se font **sur papier** ; le TP associé permet d'exécuter les mêmes templates dans Eclipse. On utilise la syntaxe **Acceleo 3** (expressions OCL).

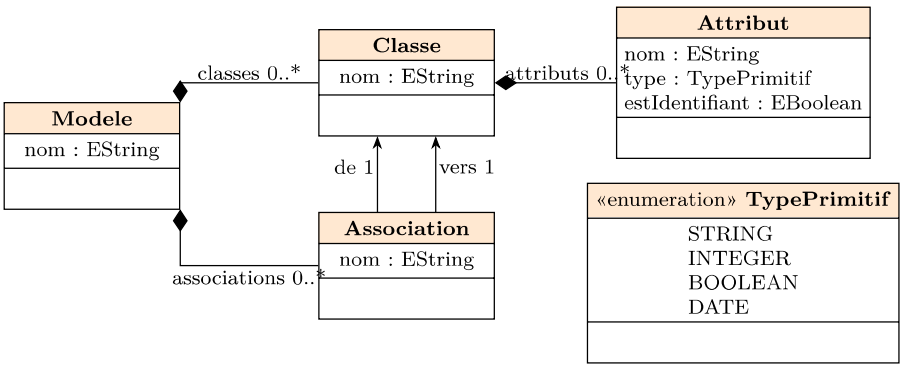


Plan du TD :

Exercice	Thème	Notion principale
1	Vrai ou faux ?	Principe du template [Simple]
2	Prédire le résultat d'un template	`[for]`, `separator`, `[if]` [Simple]
3	Naviguer dans le modèle	Expressions OCL [Simple]
4	Générer des classes Python	Compléter un template [Moyen]
5	Queries : le PDM dans le template	`[query]` [Moyen]
6	Trouver et corriger les erreurs	Syntaxe Acceleo [Moyen]
7	Générer le script SQL	Clés primaires et étrangères [Moyen]
8	Machine à états → Java	Autre méta-modèle, `oclIsTypeOf` [Avancé]
9	Du programme Java au template	Comparer les approches [Avancé]

Rappel : le méta-modèle et le modèle utilisés

Le méta-modèle `pim.ecore` (URI `http://ensias.ma/mda/pim`) :



Le modèle `bibliotheque.xmi` (un attribut sans type est de type `STRING`) :

Classe	Attributs (type)	Identifiant
Adherent	nom (STRING), email (STRING)	aucun
Livre	isbn (STRING), titre (STRING), auteur (STRING)	isbn
Emprunt	dateDebut (DATE), dateRetour (DATE), duree (INTEGER), rendu (BOOLEAN)	aucun

Associations : `emprunts` de **Adherent** vers **Emprunt** ; `emprunts` de **Livre** vers **Emprunt**.

Exercice 1 : Vrai ou faux ? [Simple]

Répondre par **vrai** ou **faux** et **corriger** les affirmations fausses.

A1. Acceleo produit un modèle conforme à un méta-modèle cible.

A2. Un template Acceleo est écrit pour **un modèle précis** (par exemple la bibliothèque).

A3. Tout le texte situé hors des crochets est recopié tel quel dans le fichier généré.

A4. L'expression `[c.nom]` écrit le nom de la classe `c`.

A5. Avec `separator(', ')`, la virgule est écrite après chaque élément, y compris le dernier.

A6. Une modification faite à la main dans un fichier généré, hors zone protégée, est perdue à la régénération.

A7. Une query renvoie une valeur ; un template renvoie un bloc de texte.

A8. Dans Acceleo 3, les expressions entre crochets sont écrites en OCL.

Exercice 2 : Prédire le résultat d'un template [Simple]

On applique le module suivant au modèle de la bibliothèque.

```
[comment encoding = UTF-8 /]
[module resume('http://ensias.ma/mda/pim')]

[template public resume(m : Modele)]
[comment @main /]
[file ('resume.txt', false, 'UTF-8')]
Modèle [m.nom/] : [m.classes->size()/] classes
[for (c : Classe | m.classes)]
- [c.nom.toUpper()/] ([for (a : Attribut | c.attributs) separator(', ')][a.nom/][if (a.estIdentifiant)
    ]*[/if][/for])
[/for]
Associations : [m.associations->size()/]
[/file]
[/template]
```

a) Combien de fichiers sont produits ? Quel est leur nom ?

b) Écrire le contenu exact du fichier généré.

c) Que produirait la ligne des classes si l'on supprimait `separator(', ')` ?

d) Quel est le rôle de `[comment @main /]` ? Que se passe-t-il si on l'oublie ?

e) Ajouter, après le nom de chaque classe, le nombre de ses attributs de type `DATE` entre crochets (ex. `EMPRUNT [2 dates]`).

Exercice 3 : Naviguer dans le modèle [Simple]

On note `m` le modèle, `l` la classe `Livre` et `e` la classe `Emprunt`. Compléter le tableau : donner la **valeur** des expressions (lignes 1 à 5) ou écrire l'**expression** demandée (lignes 6 à 10).

n°	Expression	Valeur / signification
1	`m.classes->size()`	
2	`l.attributs->first().nom.toUpperFirst()`	
3	`e.attributs->exists(a | a.estIdentifiant)`	
4	`m.associations->select(x | x.vers = e)->size()`	
5	`m.classes->collect(c | c.nom)`	
6		les attributs de type DATE de `e`
7		les classes sans attribut
8		la classe nommée « Livre »
9		les classes vers lesquelles va une association partant de `l`
10		le nom du modèle qui contient `l`

Exercice 4 : Générer des classes Python [Moyen]

On souhaite générer, à partir du PIM, un fichier `bibliotheque.py` contenant une classe Python par classe du modèle. Résultat attendu pour `Livre` et `Emprunt` :

```
from datetime import date


class Livre:
    def __init__(self, isbn: str, titre: str, auteur: str):
        self.isbn = isbn
        self.titre = titre
        self.auteur = auteur
        self.emprunts = []


class Emprunt:
    def __init__(self, dateDebut: date, dateRetour: date, duree: int, rendu: bool):
        self.dateDebut = dateDebut
        self.dateRetour = dateRetour
        self.duree = duree
        self.rendu = rendu
        self.adherent = None
        self.livre = None
```

Compléter les trous ① à ⑥ du template (la query `pythonType()` est écrite à l'exercice 5) :

```
[template public genPython(m : Modele)]
[comment @main /]
[file ( ① , false, 'UTF-8')]
from datetime import date
[for (c : Classe | m.classes)]
```

```
class [c.nom/]:
    def __init__(self[for (a : Attribut | ② )], [a.nom/]: ③ [/for]):
[for (a : Attribut | c.attributs)]
       ④
[/for]
[for (asso : Association | m.associations->select(x | ⑤ ))]
       self.[asso.nom/] = ['['/][']'/]
[/for]
[for (asso : Association | m.associations->select(x | x.vers = c))]
       ⑥
[/for]
[/for]
[/file]
[/template]
```

a) Donner le contenu des six trous.

b) Pourquoi écrit-on `['['/][']'/]` et non `[]` ?

c) La classe `Adherent` reçoit-elle un attribut `self.emprunts` ? Justifier.

d) Une classe sans attribut ni association produirait un `__init__` vide (erreur Python). Proposer une correction.

Exercice 5 : Queries – le PDM dans le template [Moyen]

a) Écrire la query `pythonType(t : TypePrimitif) : String` (STRING → `str`, INTEGER → `int`, BOOLEAN → `bool`, DATE → `date`).

b) Écrire la query `getter(a : Attribut) : String` qui renvoie `isRendu` pour un attribut booléen `rendu` et `getTitre` pour `titre`.

c) Écrire la query `cle(c : Classe) : String` qui renvoie le nom de l'identifiant de la classe, ou `'id'` s'il n'y en a pas.

d) Pourquoi dit-on que ces queries constituent une partie du **PDM** ? Que faut-il changer pour générer du **C#** au lieu de Java ?

Exercice 6 : Trouver et corriger les erreurs [Moyen]

Le module suivant (Acceleo 3) doit produire une classe Java par classe du modèle, avec ses attributs privés. Il contient **7 erreurs** : erreurs de syntaxe Acceleo, de navigation ou dans le Java produit. Les trouver et donner le module corrigé.

```
[module genJava('http://ensias.ma/mda/pim')/]

[template public genModele(m : Modele)]
[for (c : Classe | m.classe)]
[file (c.nom + '.java', false, 'UTF-8')]
public class [c.nom] {
[for (a : Attribut | c.attributs) separator(', ')]
    private [a.type.javaType()/] [a.nom/]
[/if]
}
[/for]
[/file]
[/template]
```

Exercice 7 : Générer le script SQL [Moyen]

On veut produire le fichier `bibliotheque.sql` suivant (extrait) à partir du PIM. Règles : classe → table (nom en majuscules) ; attribut → colonne ; identifiant → clé primaire, sinon colonne `id BIGINT` ; association `de` → `vers` → colonne `<de>_id` et clé étrangère dans la table `vers`. Le script commence par supprimer les tables dans l'ordre **inverse** de leur création.

```
DROP TABLE IF EXISTS EMPRUNT;
DROP TABLE IF EXISTS LIVRE;
DROP TABLE IF EXISTS ADHERENT;

CREATE TABLE ADHERENT (
    id BIGINT,
    nom VARCHAR(255),
    email VARCHAR(255),
    PRIMARY KEY (id)
);
...
CREATE TABLE EMPRUNT (
    id BIGINT,
    dateDebut DATE,
    dateRetour DATE,
    duree INTEGER,
    rendu BOOLEAN,
    adherent_id BIGINT,
    livre_id VARCHAR(255),
    PRIMARY KEY (id),
    FOREIGN KEY (adherent_id) REFERENCES ADHERENT(id),
    FOREIGN KEY (livre_id) REFERENCES LIVRE(isbn)
);
```

a) Écrire les queries `sqlType(t)` et `cleSql(c)` (type SQL de la clé primaire de `c`).

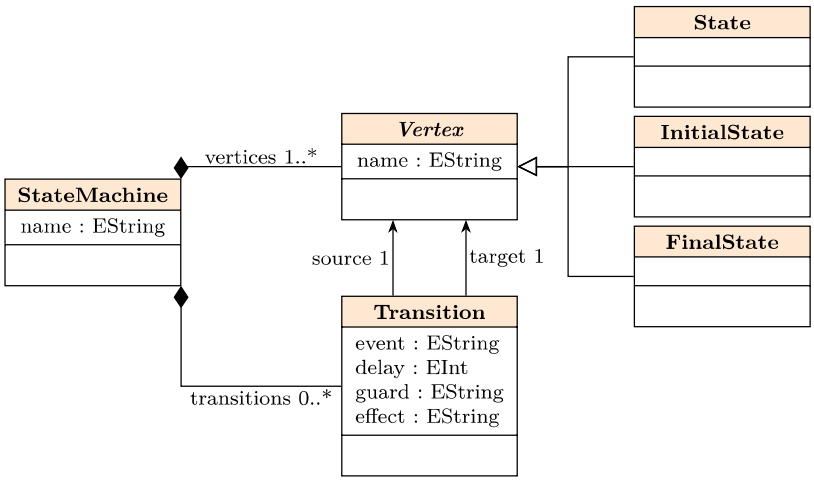
b) Écrire le template principal `genScript(m : Modele)`. Indice : `m.classes->reverse()` parcourt les classes à l'envers.

c) Pourquoi la colonne `livre_id` est-elle de type `VARCHAR(255)` et `adherent_id` de type `BIGINT` ?

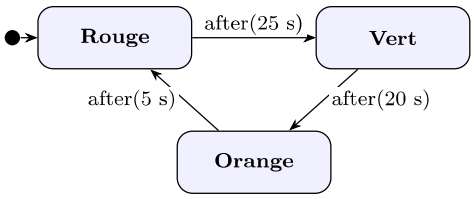
d) Pourquoi faut-il créer `ADHERENT` et `LIVRE` **avant** `EMPRUNT` ? Que se passerait-il si le modèle déclarait `Emprunt` en premier ?

Exercice 8 : Machine à états → Java [Avancé]

On change de méta-modèle : `sm.ecore` (URI `http://ensias.ma/idm/sm`) décrit des machines à états.



Le modèle `feu.xmi` décrit le feu de signalisation (état initial `debut`, état final `fin`, transition `Rouge` → `fin` sur l'événement `eteindre`) :



On veut générer :

```
public class FeuDeSignalisation {

    public enum Etat { ROUGE, VERT, ORANGE }

    private Etat courant = Etat.ROUGE;

    public void suivant() {
        switch (courant) {
            case ROUGE: courant = Etat.VERT; break; // après 25 s
            case VERT: courant = Etat.ORANGE; break; // après 20 s
            case ORANGE: courant = Etat.ROUGE; break; // après 5 s
        }
    }
}
```

a) Écrire la query `etats(sm : StateMachine)` qui renvoie les sommets de type `State` (sans l'état initial ni l'état final). Quelle est la différence entre `oclIsTypeOf` et `oclIsKindOf` ?

b) Écrire la query `etatInitial(sm)` qui renvoie l'état atteint depuis l'état initial.

c) Écrire le template `genFeu(sm : StateMachine)`. Seules les transitions entre deux `State` sont prises en compte.

d) Que suffit-il de faire pour générer le code d'un **feu piéton** (`feupieton.xmi`) ?

Exercice 9 : Du programme Java au template [Avancé]

Le programme Java suivant (approche **par programmation**, API EMF générée à partir de `pim.ecore`) produit une description XML des tables :

```
StringBuilder sb = new StringBuilder("<schema name=\"" + m.getNom() + "\">\n");
for (Classe c : m.getClasses()) {
    sb.append(" <table name=\"").append(c.getNom().toUpperCase()).append("\">\n");
    for (Attribut a : c.getAttributs()) {
        sb.append(" <column name=\"").append(a.getNom())
          .append("\" type=\"").append(a.getType()).append("\"");
        if (a.isEstIdentifiant()) sb.append(" key=\"true\"");
        sb.append("/>\n");
    }
    sb.append(" </table>\n");
}
sb.append("</schema>\n");
Files.writeString(Path.of("schema.xml"), sb.toString());
```

a) Écrire les premières lignes de `schema.xml` obtenues pour la bibliothèque (jusqu'à la table `LIVRE`).

b) Écrire le module Acceleo `genSchema.mtl` équivalent.

c) Comparer les deux approches (lisibilité, maintenance, cas d'usage).